

[Row-Level Security (RLS) and Data Permissions] [PowerBI-cheatsheet]

Part-1

1. Creating Roles and Defining Filters

- Create a new role: `CREATEROLE('Role Name')`
- Add a filter to a role: `ADDFILTER('Role Name', 'Table'[Column] = "Value")`
- Add multiple filters to a role: `ADDFILTER('Role Name', 'Table1'[Column1] = "Value1" && 'Table2'[Column2] = "Value2")`
- Add a filter with IN operator: `ADDFILTER('Role Name', 'Table'[Column] IN {"Value1", "Value2"})`
- Add a filter with NOT IN operator: `ADDFILTER('Role Name', 'Table'[Column] NOT IN {"Value1", "Value2"})`
- Add a filter with LIKE operator: `ADDFILTER('Role Name', 'Table'[Column] LIKE "A*")`
- Add a filter with wildcard: `ADDFILTER('Role Name', 'Table'[Column] = "*Value*")`
- Add a filter with greater than condition: `ADDFILTER('Role Name', 'Table'[Column] > 10)`
- Add a filter with less than condition: `ADDFILTER('Role Name', 'Table'[Column] < 20)`
- Add a filter with greater than or equal to condition: `ADDFILTER('Role Name', 'Table'[Column] >= 10)`
- Add a filter with less than or equal to condition: `ADDFILTER('Role Name', 'Table'[Column] <= 20)`
- Add a filter with between condition: `ADDFILTER('Role Name', 'Table'[Column] >= 10 && 'Table'[Column] <= 20)`

2. Implementing Dynamic Row-Level Security

- Get the current user name: `USERNAME()`
- Get the current user email: `USERPRINCIPALNAME()`
- Get the current user role: `USERROLES()`
- Check if the current user belongs to a role: `ISERRORUSERROLE("Role Name")`
- Filter data based on the current user: `'Table'[User] = USERNAME()`
- Filter data based on the current user email: `'Table'[Email] = USERPRINCIPALNAME()`
- Filter data based on the current user role: `'Table'[Role] IN USERROLES()`

- Implement dynamic row-level security: `CALCULATE('Table'[Measure], FILTER('Table', 'Table'[User] = USERNAME()))`
- Implement dynamic row-level security with multiple roles: `CALCULATE('Table'[Measure], FILTER('Table', OR('Table'[Role] = "Role1", 'Table'[Role] = "Role2")))`

3. Managing Roles and Permissions

- Delete a role: `DELETEROLE("Role Name")`
- Rename a role: `RENAMEROLE("Old Role Name", "New Role Name")`
- Get all defined roles: `ROLES()`
- Get filters for a role: `ROLEFILTERS("Role Name")`
- Check if a role exists: `ISERRORISROLE("Role Name")`
- Add a user to a role: `ADDUSERROLE("User Name", "Role Name")`
- Remove a user from a role: `REMOVEUSERROLE("User Name", "Role Name")`
- Get users in a role: `USERROLEUSERS("Role Name")`
- Check if a user belongs to a role: `ISERRORUSERINROLE("User Name", "Role Name")`

4. Applying Row-Level Security to Measures

- Apply RLS to a measure: `CALCULATE([Measure], USERROLES())`
- Apply RLS to a measure with a specific role: `CALCULATE([Measure], USERROLES("Role Name"))`
- Apply RLS to a measure with multiple roles: `CALCULATE([Measure], OR(USERROLES("Role1"), USERROLES("Role2")))`
- Apply RLS to a measure with a custom filter: `CALCULATE([Measure], FILTER('Table', 'Table'[User] = USERNAME()))`
- Apply RLS to a measure with a dynamic filter: `CALCULATE([Measure], FILTER('Table', LOOKUPVALUE('Users'[Role], 'Users'[User], USERNAME())))`

5. Handling Special Cases and Exceptions

- Allow access to all rows for a specific role: `ADDFILTER("Admin Role", 1 = 1)`
- Deny access to all rows for a specific role: `ADDFILTER("Restricted Role", 1 = 0)`
- Implement cascading RLS based on hierarchy: `ADDFILTER("Manager Role", 'Table'[ManagerID] = USERNAME())`

- Implement dynamic RLS based on user attributes: `ADDFILTER("Dynamic Role", 'Table'[Region] = LOOKUPVALUE('Users'[Region], 'Users'[User], USERNAME()))`
- Implement RLS with multiple conditions: `ADDFILTER("Complex Role", AND('Table1'[Column1] = "Value1", 'Table2'[Column2] = "Value2"))`

6. Data Permissions and Security

- Grant read permission to a table: `GRANTPERMISSION("Role Name", "Table Name", READ)`
- Grant write permission to a table: `GRANTPERMISSION("Role Name", "Table Name", READWRITE)`
- Revoke permission from a table: `REVOKEPERMISSION("Role Name", "Table Name")`
- Check if a role has read permission to a table: `ISERRORPERMISSION("Role Name", "Table Name", READ)`
- Check if a role has write permission to a table: `ISERRORPERMISSION("Role Name", "Table Name", READWRITE)`
- Get tables with read permission for a role: `PERMISSIONTABLES("Role Name", READ)`
- Get tables with write permission for a role: `PERMISSIONTABLES("Role Name", READWRITE)`

7. Auditing and Monitoring

- Log RLS changes: `SELECTCOLUMNS(USAGEDETAILS(), "User", [User], "Object", [Object], "Action", [Action])`
- Track user activity: `RANKX(ALLSELECTED('Table'), COUNTROWS(FILTER('Table', 'Table'[User] = EARLIER('Table'[User]))))`
- Monitor RLS performance: `EVALUATE(ROW("Query Time", FORMAT(AVERAGEX(USAGEDETAILS(), [Duration]), "hh:mm:ss")))`
- Audit data access: `SELECTCOLUMNS(USAGEDETAILS(), "User", [User], "Table", [Object], "Rows Accessed", [RowsAccessed])`
- Identify frequently accessed tables: `TOPN(10, SUMMARIZECOLUMNS(USAGEDETAILS()[Object], "Table", [Object], "Access Count", COUNTROWS(CURRENTGROUP())))`

8. Integration with Other Security Mechanisms

- Integrate with Active Directory: `LOOKUPVALUE('Users'[Role], 'Users'[User], USERPRINCIPALNAME())`

- Integrate with Azure Active Directory: `LOOKUPVALUE('Users'[Role], 'Users'[Email], USERPRINCIPALNAME())`
- Integrate with custom security tables: `LOOKUPVALUE('SecurityTable'[Role], 'SecurityTable'[User], USERNAME())`
- Implement single sign-on (SSO) with RLS: `LOOKUPVALUE('Users'[Role], 'Users'[Email], USERPRINCIPALNAME())`
- Implement multi-factor authentication (MFA) with RLS:
`AND(ISERRORUSERINROLE(USERNAME(), "Authenticated Role"),`
`ISERRORUSERINROLE(USERNAME(), "MFA Role"))`

9. Performance Optimization

- Use `USERMODE` for efficient RLS evaluation: `CALCULATE([Measure], USERMODE("Role1", "Role2"))`
- Avoid using `LOOKUPVALUE` in RLS filters: `ADDFILTER("Role", 'Table'[Column] = "Value")`
- Minimize the number of RLS roles: `REDUCE(0, ROLES(), COUNTROWS(ROLEFILTERS(_)))`
- Use `SUMMARIZE` instead of `ADDCOLUMNS` for RLS filters: `ADDFILTER("Role", SUMMARIZE('Table', 'Table'[Column]))`
- Avoid using complex calculations in RLS filters: `ADDFILTER("Role", 'Table'[Column] = "Value")`

10. Advanced RLS Techniques

- Implement dynamic row-level security with measures: `CALCULATE([Measure], FILTER('Table', [Dynamic RLS Measure] = "Value"))`
- Implement column-level security: `CALCULATE([Measure], REMOVEFILTERS('Table'[Restricted Column]))`
- Implement data masking: `IF(ISERRORUSERINROLE(USERNAME(), "Privileged Role"), 'Table'[Column], "*****")`
- Implement time-based access control: `ADDFILTER("Role", 'Table'[Date] >= DATE(YEAR(TODAY()), MONTH(TODAY()), 1) && 'Table'[Date] < DATE(YEAR(TODAY()), MONTH(TODAY()) + 1, 1))`
- Implement data-driven permissions: `ADDFILTER("Role", 'Table'[Access Level] <= LOOKUPVALUE('Users'[Access Level], 'Users'[User], USERNAME()))`
- Implement RLS with many-to-many relationships: `ADDFILTER("Role", 'Bridge Table'[User] = USERNAME() && 'Bridge Table'[Object] = 'Table'[Object])`

11. Troubleshooting and Error Handling

- Enable tracing for RLS issues: `TRACE(TRUE, "RLS Trace")`
- Capture RLS errors: `IFERROR(CALCULATE([Measure], USERROLES()), "RLS Error")`
- Display user-friendly error messages: `IF(ISERRORUSERROLE(USERNAME()), "Role"), [Measure], "Access Denied")`
- Investigate RLS performance issues: `EVALUATE(ROW("RLS Evaluation Time", FORMAT(AVERAGEX(DETAILROWS(USAGEDETAILS()), [Duration]), "hh:mm:ss")))`
- Diagnose RLS filter conflicts: `CALCULATETABLE(VALUES('Table'[Column]), INTERSECT(USERROLES("Role1"), USERROLES("Role2")))`

Part-2

1. Creating Roles

- Create a new role: `Manage Roles > Create Role`
- Name a role: `Roles > Rolename > Name = "Role Name"`
- Set role description: `Roles > Rolename > Description = "Role Description"`

2. Defining Row Filters

- Add a table filter: `Roles > Rolename > Tables > Tablename > Add Filter`
- Define a DAX filter: `Roles > Rolename > Tables > Tablename > Table Filter = [ColumnName] = "FilterValue"`
- Use `USERPRINCIPALNAME()` function: `Roles > Rolename > Tables > Tablename > Table Filter = [ColumnName] = USERPRINCIPALNAME()`
- Use `USERNAME()` function: `Roles > Rolename > Tables > Tablename > Table Filter = [ColumnName] = USERNAME()`
- Use `CUSTOMDATA()` function: `Roles > Rolename > Tables > Tablename > Table Filter = [ColumnName] = CUSTOMDATA()`
- Use AND operator: `Roles > Rolename > Tables > Tablename > Table Filter = [ColumnName1] = "Value1" && [ColumnName2] = "Value2"`
- Use OR operator: `Roles > Rolename > Tables > Tablename > Table Filter = [ColumnName1] = "Value1" || [ColumnName2] = "Value2"`
- Use IN operator: `Roles > Rolename > Tables > Tablename > Table Filter = [ColumnName] IN ("Value1", "Value2", "Value3")`
- Use NOT IN operator: `Roles > Rolename > Tables > Tablename > Table Filter = [ColumnName] NOT IN ("Value1", "Value2", "Value3")`
- Use CONTAINS operator: `Roles > Rolename > Tables > Tablename > Table Filter = CONTAINS([ColumnName], "Value")`

- Use STARTSWITH operator: Roles > Rolename > Tables > Tablename > Table Filter = STARTSWITH([ColumnName], "Prefix")
- Use ENDSWITH operator: Roles > Rolename > Tables > Tablename > Table Filter = ENDSWITH([ColumnName], "Suffix")

3. Applying Row Filters

- Apply filter to all rows: Roles > Rolename > Tables > Tablename > Row Filters > Apply to all rows
- Exclude specific columns: Roles > Rolename > Tables > Tablename > Row Filters > Exclude columns > Select columns to exclude
- Exclude specific measures: Roles > Rolename > Tables > Tablename > Row Filters > Exclude measures > Select measures to exclude
- Apply filter to related tables: Roles > Rolename > Tables > Tablename > Row Filters > Apply filter to related tables

4. Managing Roles

- Add a member to a role: Roles > Rolename > Members > Add member
- Remove a member from a role: Roles > Rolename > Members > Remove member
- Clone a role: Roles > Rolename > Clone Role
- Delete a role: Roles > Rolename > Delete Role

5. Testing Roles

- View as a role: Modeling > View as Roles > Select Role
- View as a specific user: Modeling > View as Roles > Other User > Enter User Name or Email
- Revert to default view: Modeling > View as Roles > Default

6. Dynamic Row-Level Security

- Use USERPRINCIPALNAME() in a measure: Measure = IF(USERPRINCIPALNAME() = "user@domain.com", [ColumnName], BLANK())
- Use USERNAME() in a measure: Measure = IF(USERNAME() = "Domain\Username", [ColumnName], BLANK())
- Use CUSTOMDATA() in a measure: Measure = IF(CUSTOMDATA() = "CustomValue", [ColumnName], BLANK())
- Use a measure in a role filter: Roles > Rolename > Tables > Tablename > Table Filter = [Measure] = "FilterValue"

7. Data Source Permissions

- Restrict access to a data source: Data Source Settings > Permissions > Add User or Group
- Grant read permission: Data Source Settings > Permissions > Read > Assign User or Group
- Grant write permission: Data Source Settings > Permissions > Write > Assign User or Group
- Grant admin permission: Data Source Settings > Permissions > Admin > Assign User or Group

8. Implementing Column-Level Security

- Hide a column: Table > Column > Hide
- Hide multiple columns: Table > Select Columns > Hide
- Grant permission to a column: Table > Column > Security > Add User or Group
- Revoke permission from a column: Table > Column > Security > Remove User or Group

9. Securing Measures

- Hide a measure: Measure > Hide
- Hide multiple measures: Select Measures > Hide
- Grant permission to a measure: Measure > Security > Add User or Group
- Revoke permission from a measure: Measure > Security > Remove User or Group

10. Securing Hierarchies

- Hide a hierarchy: Hierarchy > Hide
- Hide multiple hierarchies: Select Hierarchies > Hide
- Grant permission to a hierarchy: Hierarchy > Security > Add User or Group
- Revoke permission from a hierarchy: Hierarchy > Security > Remove User or Group

11. Securing Calculated Columns

- Hide a calculated column: Calculated Column > Hide
- Hide multiple calculated columns: Select Calculated Columns > Hide

- Grant permission to a calculated column: `Calculated Column > Security > Add User or Group`
- Revoke permission from a calculated column: `Calculated Column > Security > Remove User or Group`

12. Securing Calculated Tables

- Hide a calculated table: `Calculated Table > Hide`
- Hide multiple calculated tables: `Select Calculated Tables > Hide`
- Grant permission to a calculated table: `Calculated Table > Security > Add User or Group`
- Revoke permission from a calculated table: `Calculated Table > Security > Remove User or Group`

13. Securing Relationships

- Hide a relationship: `Relationship > Hide`
- Hide multiple relationships: `Select Relationships > Hide`
- Grant permission to a relationship: `Relationship > Security > Add User or Group`
- Revoke permission from a relationship: `Relationship > Security > Remove User or Group`

14. Securing Report Pages

- Hide a report page: `Report Page > Hide`
- Hide multiple report pages: `Select Report Pages > Hide`
- Grant permission to a report page: `Report Page > Security > Add User or Group`
- Revoke permission from a report page: `Report Page > Security > Remove User or Group`

15. Securing Visuals

- Hide a visual: `Visual > Hide`
- Hide multiple visuals: `Select Visuals > Hide`
- Grant permission to a visual: `Visual > Security > Add User or Group`
- Revoke permission from a visual: `Visual > Security > Remove User or Group`

16. Securing Drill-Through Actions

- Disable drill-through for a visual: Visual > Drill-Through > Disable
- Enable drill-through for a visual: Visual > Drill-Through > Enable
- Set drill-through destination: Visual > Drill-Through > Destination > Select Destination Page
- Set drill-through fields: Visual > Drill-Through > Fields > Select Drill-Through Fields

17. Securing Bookmarks

- Hide a bookmark: Bookmark > Hide
- Hide multiple bookmarks: Select Bookmarks > Hide
- Grant permission to a bookmark: Bookmark > Security > Add User or Group
- Revoke permission from a bookmark: Bookmark > Security > Remove User or Group

18. Securing Buttons

- Hide a button: Button > Hide
- Hide multiple buttons: Select Buttons > Hide
- Grant permission to a button: Button > Security > Add User or Group
- Revoke permission from a button: Button > Security > Remove User or Group

19. Securing Tooltips

- Disable tooltips for a visual: Visual > Tooltips > Disable
- Enable tooltips for a visual: Visual > Tooltips > Enable
- Set tooltip fields: Visual > Tooltips > Fields > Select Tooltip Fields
- Set tooltip measures: Visual > Tooltips > Measures > Select Tooltip Measures

20. Securing Slicers

- Hide a slicer: Slicer > Hide
- Hide multiple slicers: Select Slicers > Hide
- Grant permission to a slicer: Slicer > Security > Add User or Group
- Revoke permission from a slicer: Slicer > Security > Remove User or Group

21. Securing Quick Insights

- Disable Quick Insights: File > Options and Settings > Options > Quick Insights > Disable
- Enable Quick Insights: File > Options and Settings > Options > Quick Insights > Enable
- Set Quick Insights fields: File > Options and Settings > Options > Quick Insights > Fields > Select Fields
- Set Quick Insights measures: File > Options and Settings > Options > Quick Insights > Measures > Select Measures

22. Securing Q&A

- Disable Q&A: File > Options and Settings > Options > Q&A > Disable
- Enable Q&A: File > Options and Settings > Options > Q&A > Enable
- Set Q&A fields: File > Options and Settings > Options > Q&A > Fields > Select Fields
- Set Q&A measures: File > Options and Settings > Options > Q&A > Measures > Select Measures

23. Securing Export Options

- Disable export to Excel: File > Options and Settings > Options > Export > Excel > Disable
- Enable export to Excel: File > Options and Settings > Options > Export > Excel > Enable
- Disable export to CSV: File > Options and Settings > Options > Export > CSV > Disable
- Enable export to CSV: File > Options and Settings > Options > Export > CSV > Enable
- Disable export to PDF: File > Options and Settings > Options > Export > PDF > Disable
- Enable export to PDF: File > Options and Settings > Options > Export > PDF > Enable

#. Best Practices and Guidelines

- Keep RLS roles and filters simple and manageable
- Use meaningful names for roles and tables
- Test RLS thoroughly with different user contexts
- Avoid using sensitive information in RLS filters

- Regularly review and update RLS roles and permissions
- Educate users about data security and access policies
- Implement a robust access control matrix
- Follow the principle of least privilege
- Use a centralized security model for consistency
- Regularly audit and monitor RLS usage and performance